p8106_hw3

Charlotte Zhang

2026-03-30

```
# Import data
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   4.0.0     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidymodels)
```

```
## -- Attaching packages --------------------------------------- tidymodels 1.4.1 --
## v broom        1.0.9     v rsample      1.3.2
## v dials        1.4.2     v tailor       0.1.0
## v infer        1.0.9     v tune         2.0.1
## v modeldata    1.5.1     v workflows    1.3.0
## v parsnip      1.4.1     v workflowsets 1.1.1
## v recipes      1.3.1     v yardstick    1.3.2

## Warning: package 'parsnip' was built under R version 4.5.2

## Warning: package 'rsample' was built under R version 4.5.2

## -- Conflicts ------------------------------------------ tidymodels_conflicts() --
## x scales::discard() masks purrr::discard()
## x dplyr::filter()   masks stats::filter()
## x recipes::fixed()  masks stringr::fixed()
## x dplyr::lag()      masks stats::lag()
## x yardstick::spec() masks readr::spec()
## x recipes::step()   masks stats::step()
```

```
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following objects are masked from 'package:yardstick':
##
##     precision, recall, sensitivity, specificity
##
## The following object is masked from 'package:rsample':
##
##     calibration
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(earth)
```

```
## Warning: package 'earth' was built under R version 4.5.2

## Loading required package: Formula
## Loading required package: plotmo

## Warning: package 'plotmo' was built under R version 4.5.2

## Loading required package: plotrix

## Warning: package 'plotrix' was built under R version 4.5.2

##
## Attaching package: 'plotrix'
##
## The following object is masked from 'package:scales':
##
##     rescale
```

```
library(MASS)
```

```
##
## Attaching package: 'MASS'
##
## The following object is masked from 'package:dplyr':
##
##     select
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
##
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```r
library(vip)
```

```
## Warning: package 'vip' was built under R version 4.5.2
```

```
##
## Attaching package: 'vip'
##
## The following object is masked from 'package:utils':
##
##     vi
```

```r
library(pdp)
```

```
## Warning: package 'pdp' was built under R version 4.5.2
```

```
##
## Attaching package: 'pdp'
##
## The following object is masked from 'package:purrr':
##
##     partial
```

```r
auto <- read.csv("auto.csv")

auto$mpg_cat <- as.factor(auto$mpg_cat)
auto$origin <- as.numeric(auto$origin)
```

```r
# train test split
set.seed(1)
data_split <- initial_split(auto, prop = 0.7, strata = mpg_cat)
training_data <- training(data_split)
testing_data  <- testing(data_split)

training_data$mpg_cat <- factor(training_data$mpg_cat,
                                levels = c("low", "high"))

testing_data$mpg_cat <- factor(testing_data$mpg_cat,
                               levels = c("low", "high"))
```

```r
# cross validation
ctrl <- trainControl(
  method = "cv",
  number = 10,
  summaryFunction = twoClassSummary,
  classProbs = TRUE
)
```

Problem a

```r
glm.fit <- glm(mpg_cat ~ .,
               data = training_data,
               family = binomial(link = "logit"))
summary(glm.fit)
```

```
##
## Call:
## glm(formula = mpg_cat ~ ., family = binomial(link = "logit"),
##     data = training_data)
##
## Coefficients:
##                Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -3.060e+01  8.019e+00  -3.816 0.000135 ***
## cylinders     1.804e-01  5.537e-01   0.326 0.744522
## displacement  3.633e-04  1.557e-02   0.023 0.981391
## horsepower   -2.346e-02  2.913e-02  -0.805 0.420591
## weight       -5.035e-03  1.420e-03  -3.546 0.000391 ***
## acceleration  9.050e-02  1.670e-01   0.542 0.587800
## year          5.677e-01  1.069e-01   5.313 1.08e-07 ***
## origin        9.237e-01  4.701e-01   1.965 0.049425 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 379.84  on 273  degrees of freedom
## Residual deviance: 107.17  on 266  degrees of freedom
## AIC: 123.17
##
## Number of Fisher Scoring iterations: 8
```

```r
# train logistic regression model with CV
set.seed(1)
model.glm <- train(x = training_data[, names(training_data) != "mpg_cat"],
y = training_data$mpg_cat,
method = "glm",
metric = "ROC",
trControl = ctrl)
```

Predictors with a large p-value such as cylinders, displacement, horsepower, acceleration might be redundant. They are not statistically significant after controlling for the others, since they have a p-value greater than 0.05, possibly due to multicollinearity or overlapping information.

Problem b

```r
# train MARS
set.seed(1)
model.mars <- train(
  x = training_data[, names(training_data) != "mpg_cat"],
```

```
  y = training_data$mpg_cat,
  method = "earth",
  tuneGrid = expand.grid(
    degree = 1:4,
    nprune = 2:20
  ),
  metric = "ROC",
  trControl = ctrl
)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

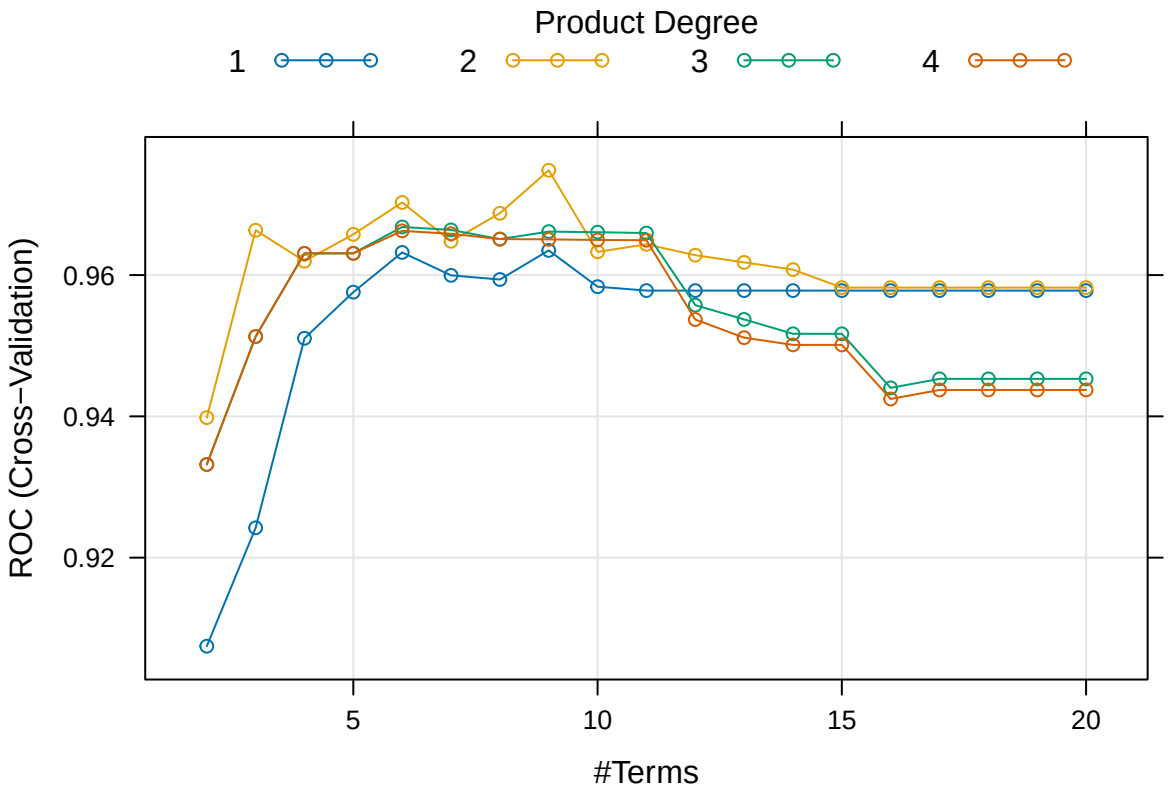
```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

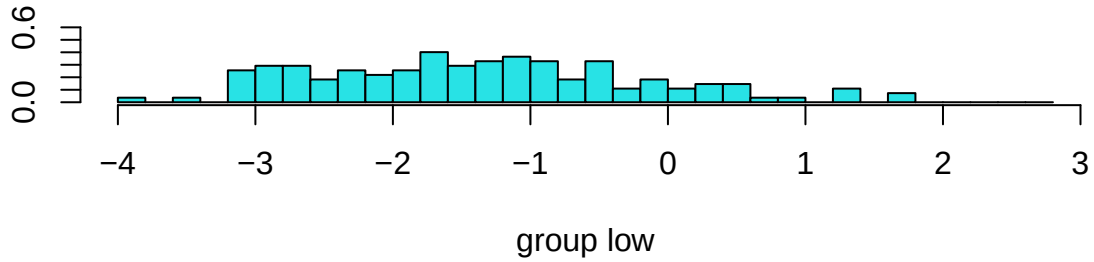
```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
plot(model.mars)
```

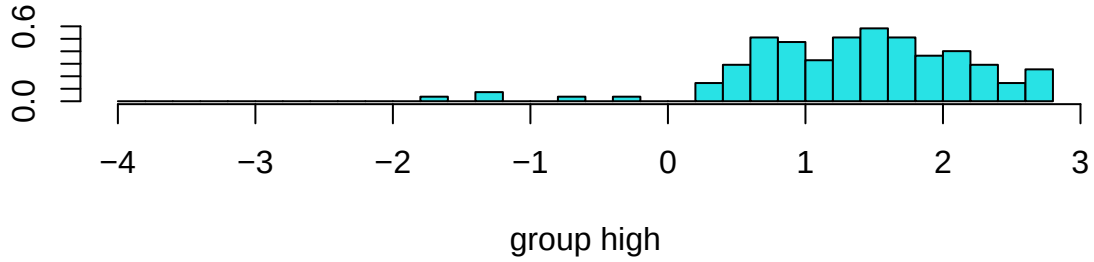


Problem c

```
lda.fit <- lda(mpg_cat ~ ., data = training_data)
# plot discriminant
plot(lda.fit)
```



group low



group high

```
# show coefficients
lda.fit$scaling
```

```
##                       LD1
## cylinders    -0.293313306
## displacement -0.001432553
## horsepower    0.013350630
## weight       -0.001331618
## acceleration  0.012122163
## year          0.157741602
## origin        0.287676861
```

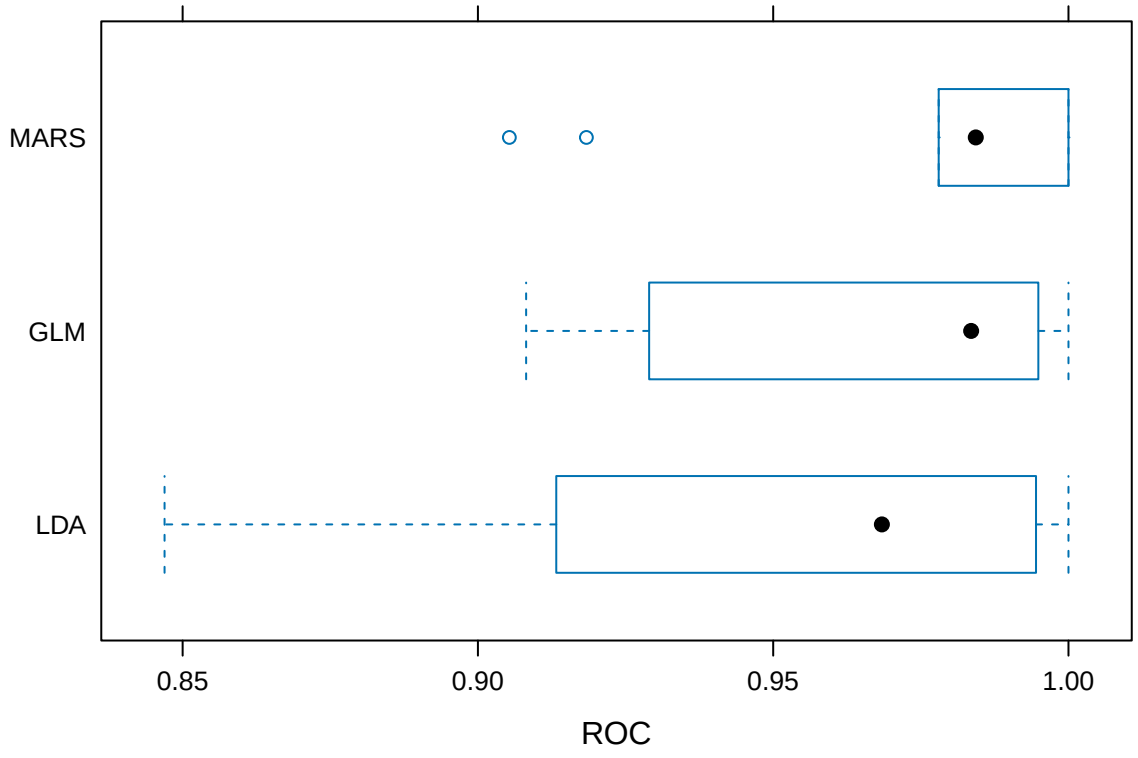
```
# Cross validation LDA
set.seed(1)
model.lda <- train(
  x = training_data[, names(training_data) != "mpg_cat"],
  y = training_data$mpg_cat,
  method = "lda",
  metric = "ROC",
  trControl = ctrl
)
```

Problem d

```
# compare models using resampling
res <- resamples(list(GLM = model.glm,
                      LDA = model.lda,
                      MARS = model.mars))
summary(res)
```

```
##
## Call:
## summary.resamples(object = res)
##
## Models: GLM, LDA, MARS
## Number of resamples: 10
##
## ROC
##           Min.  1st Qu.    Median      Mean   3rd Qu. Max. NA's
## GLM  0.9081633 0.937817 0.9835165 0.9681349 0.9947998    1    0
## LDA  0.8469388 0.923960 0.9684066 0.9502083 0.9920526    1    0
## MARS 0.9053254 0.978022 0.9843014 0.9748340 1.0000000    1    0
##
## Sens
##           Min.   1st Qu.    Median      Mean   3rd Qu. Max. NA's
## GLM  0.6923077 0.8571429 0.9258242 0.8972527 0.9821429    1    0
## LDA  0.4615385 0.8008242 0.8571429 0.8521978 0.9821429    1    0
## MARS 0.7692308 0.8750000 0.9642857 0.9340659 1.0000000    1    0
##
## Spec
##           Min.   1st Qu.    Median      Mean   3rd Qu. Max. NA's
## GLM  0.8461538 0.8571429 0.9258242 0.9197802 0.9821429    1    0
## LDA  0.9230769 0.9285714 0.9642857 0.9631868 1.0000000    1    0
## MARS 0.8461538 0.9230769 0.9285714 0.9406593 1.0000000    1    0
```

```
# boxplot of ROC performance for all models
bwplot(res, metric = "ROC")
```




```
# predictions on test data for each model
glm.pred  <- predict(model.glm,  newdata = testing_data, type = "prob")[,2]
mars.pred <- predict(model.mars, newdata = testing_data, type = "prob")[,2]
lda.pred  <- predict(model.lda,  newdata = testing_data, type = "prob")[,2]


roc.glm  <- roc(testing_data$mpg_cat, glm.pred)
```

```
## Setting levels: control = low, case = high
```

```
## Setting direction: controls < cases
```

```
roc.mars <- roc(testing_data$mpg_cat, mars.pred)
```

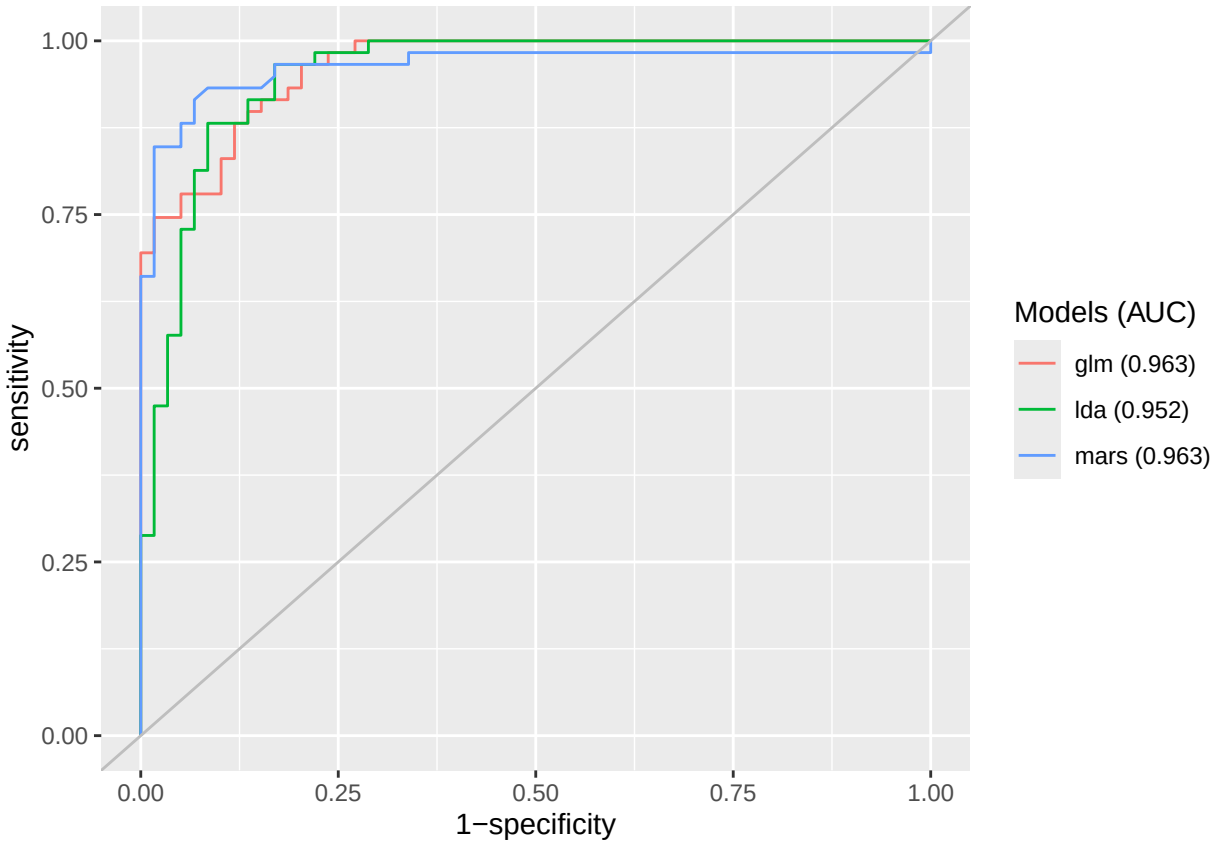
```
## Setting levels: control = low, case = high
## Setting direction: controls < cases
```

```
roc.lda  <- roc(testing_data$mpg_cat, lda.pred)
```

```
## Setting levels: control = low, case = high
## Setting direction: controls < cases
```

```r
auc <- c(roc.glm$auc[1], roc.lda$auc[1], roc.mars$auc[1])
```

```r
# plot ROC curves for all models
modelNames <- c("glm","lda","mars")
ggroc(list(roc.glm, roc.lda, roc.mars), legacy.axes = TRUE) +
scale_color_discrete(labels = paste0(modelNames, " (", round(auc,3),")"),
name = "Models (AUC)") +
geom_abline(intercept = 0, slope = 1, color = "grey")
```



Problem e

```r
final.prob <- mars.pred

final.pred <- rep(levels(training_data$mpg_cat)[1], length(final.prob))
final.pred[final.prob > 0.5] <- levels(training_data$mpg_cat)[2]

final.pred <- as.factor(final.pred)

confusionMatrix(
  data = final.pred,
  reference = testing_data$mpg_cat,
  positive = levels(training_data$mpg_cat)[2]
)
```

```
## Warning in confusionMatrix.default(data = final.pred, reference =
## testing_data$mpg_cat, : Levels are not in the same order for reference and
## data. Refactoring data to match.
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction low high
##       low   53    4
##       high   6   55
##
##                Accuracy : 0.9153
##                  95% CI : (0.8497, 0.9586)
##     No Information Rate : 0.5
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.8305
##
##  Mcnemar's Test P-Value : 0.7518
##
##             Sensitivity : 0.9322
##             Specificity : 0.8983
##          Pos Pred Value : 0.9016
##          Neg Pred Value : 0.9298
##              Prevalence : 0.5000
##          Detection Rate : 0.4661
##    Detection Prevalence : 0.5169
##       Balanced Accuracy : 0.9153
##
##        'Positive' Class : high
##
```